From Mistake Detection to Task Verification: A Multi-Stage Pipeline for Procedural Activity Understanding

Politecnico di Torino
s123456@studenti.polito.it

January 24, 2026

Abstract

We present a study on procedural mistake detection and task verification using the CaptainCook4D dataset. We reproduce and analyze baseline error recognition models and develop a refactored, end-to-end extension pipeline: (1) temporal step localization with ActionFormer, (2) recording-level task verification from step embeddings, (3) task graph matching via Hungarian assignment, and (4) graph classification with GNNs. In our current implementation, ActionFormer step segmentation reaches a greedy segment overlap of IoU ≈ 0.603 on the test split when compared to ground-truth step boundaries (with boundary F1@$\pm$2s ≈ 0.256). For recipe-level verification, GraphSAGE on realized task graphs achieves F1 $= 75.75\% \pm 10.66\%$ and AUC $= 82.25\% \pm 9.52\%$ under leave-one-recipe-out evaluation; by tuning the operating point (decision threshold / positive-class weighting) for recall-first mistake detection, we reach F1 $\approx 78.09\%$ with recall ≈ 0.848. We summarize the main insights and the tradeoffs between recall-first and precision-first operating points.

1 Introduction

Procedural activity understanding is a fundamental challenge in computer vision that involves teaching models to recognize, segment, and reason about sequences of actions in complex human activities. A particularly important application is mistake detection—identifying errors in procedural executions such as cooking recipes, assembly tasks, or medical procedures.

In this project, we focus on the CaptainCook4D dataset [1], which provides 384 egocentric videos of recipe executions with various intentional and unintentional errors. Our goal is twofold: first, to implement and analyze baseline models for step-level mistake detection, and second, to develop an end-to-end pipeline for recipe-level task verification.

The task verification problem is more challenging than step-level mistake detection because it requires understanding the entire recipe execution in context. We approach this through a four-stage pipeline: (1) temporal action localization to detect recipe steps, (2) step-level verification using simple baselines, (3) task graph matching to align detected steps with the recipe structure, and (4) graph neural network classification to predict overall recipe correctness.

Our main contributions include:

- Reproduction and analysis of V1 (MLP) and V2 (Transformer) baselines from CaptainCook4D [1], with a novel V3 (LSTM) baseline achieving 58.41% F1
- Analysis of model performance across different error types (TechniqueError, TimingError, etc.)
- Extensive hyperparameter analysis of ActionFormer for temporal localization, with detailed explanation of why performance is inherently limited on this dataset
- A counter-intuitive "regularization paradox" where stronger regularization hurts larger models
- A complete task verification pipeline with task graph matching and GNN classification achieving 75.75% F1 (GraphSAGE, LORO), and improving to $\approx$78.09% F1 under recall-first operating point tuning (recall $\approx$0.848)

2 Related Work

2.1 CaptainCook4D and Supervised Error Recognition

Our work builds directly upon CaptainCook4D [1], a dataset of 384 egocentric cooking videos with fine-grained error annotations. The original paper proposes three baselines for the Supervised Error Recognition (SupervisedER) task: **V1** is a simple Multi-Layer Perceptron

(MLP) that processes averaged sub-segment features; **V2** uses a Transformer encoder to model relationships between sub-segments within a step; **V3** extends V2 with multimodal inputs (audio, depth, narrations). In this work, we reproduce V1 and V2, and propose an alternative V3 using LSTM to capture temporal dependencies.

2.2 Procedural Activity Understanding

Recent work has made significant progress in understanding procedural activities. Seminara et al. [2] propose differentiable task graph learning for online mistake detection from egocentric videos, learning task graphs directly from demonstrations. Flaborea et al. [3] introduce PREGO for *online* mistake detection in procedural egocentric videos. Unlike these online approaches, our work focuses on *offline* detection where the entire video is available. HiERO [4] enhances reasoning by understanding the hierarchy of human behavior through clustering-based step discovery.

2.3 Video Feature Extraction

Modern approaches rely on pre-trained video backbones for feature extraction. Omnivore [5] provides a single model for many visual modalities, while SlowFast [6] separates slow and fast pathways for temporal modeling. EgoVLP [7] provides aligned video-language embeddings particularly suited for egocentric video understanding. Following CaptainCook4D, we use Omnivore and SlowFast features for baseline reproduction, and EgoVLP features for the extension pipeline where text-video alignment is required.

2.4 Temporal Action Localization

ActionFormer [8] is a state-of-the-art approach for temporal action localization using transformer-based architectures with local self-attention and multi-scale feature pyramids. Pre-trained ActionFormer models are typically trained on large-scale datasets like Ego4D or ActivityNet. However, no pre-trained checkpoint exists for CaptainCook4D, requiring training from scratch—a significant challenge given the dataset's limited size (384 videos) and fine-grained cooking step categories.

2.5 Graph Neural Networks

GNNs have been applied to structured prediction tasks. DAGNN [9] is specifically designed for directed acyclic graphs, making it suitable for task graph reasoning. We experiment with GCN [10], GAT [11], and GraphSAGE [12] for classifying realized task graphs.

3 Method

Our approach consists of four main components as illustrated in Figure 1.

3.1 Step 1: Temporal Action Localization

We implement ActionFormer [8] for localizing recipe steps in untrimmed videos. The architecture consists of:

Backbone: A ConvTransformerBackbone with multiple stages of convolution and local masked multi-head attention (LocalMaskedMHCA). Each stage applies strided convolution followed by self-attention with a fixed window size.

Neck: A Feature Pyramid Network (FPN1D) that builds multi-scale representations by combining features from different backbone stages.

Head: Classification and regression heads that predict action classes and temporal boundaries at each scale.

The model is trained with a combination of focal loss for classification and DIoU loss for regression:

$$\mathcal{L} = \mathcal{L}_{focal} + \lambda_{reg}\mathcal{L}_{DIoU} \tag{1}$$

We use EgoVLP features (256-dimensional) as input, which provide aligned video-language representations suitable for matching with text descriptions.

3.2 Baseline Error Recognition (V1, V2, V3)

Following CaptainCook4D [1], we implement the SupervisedER baselines for step-level mistake detection:

V1 - MLP Baseline: A two-layer MLP that processes averaged sub-segment features within each step:

$$h = \text{ReLU}(\text{Linear}(x)), \quad \hat{y} = \sigma(\text{Linear}(\text{Dropout}(h))) \tag{2}$$

V2 - Transformer Baseline: A transformer encoder that models relationships between sub-segment features within a step, followed by mean pooling and a classification head. This allows the model to capture interactions between different temporal segments.

V3 - LSTM Baseline (Ours): We propose a bidirectional LSTM as an alternative to the multimodal V3 from the original paper:

$$h_t = \text{BiLSTM}(x_t, h_{t-1}) \tag{3}$$

The LSTM explicitly models temporal dependencies in the sub-segment sequence, which we hypothesize is important for detecting procedural errors that manifest over time.

pipeline.png missing
(replace with your pipeline figure)

Figure 1: Overview of our task verification pipeline. (1) ActionFormer localizes recipe steps from video features. (2) Simple baselines verify individual steps. (3) Hungarian matching aligns detected steps with task graph nodes. (4) GNN classifier predicts recipe correctness.

3.3 Task Verification Baselines (Extension)

For the extension's task verification step, we adapt the same architectures (MLP, Transformer, LSTM) to process step-level embeddings extracted from ActionFormer or ground-truth boundaries, predicting whether the overall recipe execution is correct.

3.4 Step 3: Task Graph Matching

Given detected steps from ActionFormer and the ground-truth task graph, we match each visual step to a graph node using Hungarian matching:

$$\mathbf{M}^* = \arg\min_{\mathbf{M}} \sum_{i,j} \mathbf{M}_{ij} \cdot (1 - \text{sim}(v_i, g_j)) \quad (4)$$

where v_i is the visual embedding of step i, g_j is the text embedding of graph node j, and $\text{sim}(\cdot, \cdot)$ is cosine similarity. The aligned visual-text space of EgoVLP enables this cross-modal matching.

After matching, we create a "realized" task graph by updating matched nodes with fused features:

$$g'_j = \text{Proj}([g_j; v_i]) \quad (5)$$

Implementation note (text embeddings): While the ideal setup uses the aligned EgoVLP/PerceptionEncoder text encoder, some of our local runs use a lightweight alternative to avoid downloading large checkpoints: we encode node descriptions with a small text model and fit a ridge-regression projection into the Step-1 visual embedding space using GT pairs $(\text{description}, v)$. The learned projection is cached (e.g., as `extension_data/step3_text_to_visual_W.npy`) and allows Hungarian matching to operate in the same space as Step 1.

3.5 Step 4: GNN Classification

We train a Graph Neural Network to classify the realized task graph as correct or incorrect. We experiment with multiple GNN architectures:

GCN: Graph Convolutional Networks [10]

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}) \quad (6)$$

GAT: Graph Attention Networks [11] with attention-based message passing

GraphSAGE: Sampling and aggregation for inductive learning [12]

After GNN layers, we apply global mean pooling and a classification head:

$$\hat{y} = \sigma(\text{Linear}(\text{MeanPool}(H^{(L)}))) \quad (7)$$

4 Experiments

4.1 Dataset and Setup

We use the CaptainCook4D dataset [1] containing 384 egocentric cooking videos across multiple recipes. Each video contains step-level annotations with labels indicating correct or incorrect execution.

Dataset Statistics: The dataset includes recordings from multiple environments ("environment" split), multiple people ("person" split), and across different recording sessions ("recordings" split). We primarily use the "step" and "recordings" splits following the original paper. The dataset contains five error categories: TechniqueError, PreparationError, MeasurementError, TimingError, and TemperatureError.

Features: For baseline reproduction (Section 4.2), we use pre-extracted Omnivore (1024-dim) and SlowFast (2304-dim) features as in the original paper. For the extension pipeline, we use EgoVLP features (256-dim) which provide aligned video-text embeddings required for task graph matching.

Evaluation Protocol: Following CaptainCook4D, we use different classification thresholds for different splits: **threshold=0.6** for the step split and **threshold=0.5** for the recordings split. We report Precision, Recall, F1-score, and AUC. For GNN experiments, we use leave-one-recipe-out cross-validation.

Important Reproducibility Note: The CaptainCook4D paper [1] specifies threshold=0.4 for the recordings split, but through our reproducibility experiments, we discovered that threshold=0.5 produces results consistent with those reported in the paper. We confirmed this discrepancy with the teaching assistants and verified that threshold=0.5 is the correct value for replicating the published results. This finding is documented in the official repository issues as well.

4.2 Baseline Reproduction Results

We reproduce V1 (MLP) and V2 (Transformer) baselines from CaptainCook4D [1]. To establish ground truth for comparison, we verified the official baselines using the authors' provided checkpoints, then trained our proposed V3 (LSTM) from scratch using the same training configuration to ensure fair comparison. Table 1 shows results using Omnivore features.

Key Finding: Our LSTM (V3) achieves competitive performance with the Transformer baseline (V2), while significantly outperforming both V1 and V2 on the recordings split (58.41% vs 50.91% and 39.04%). The LSTM's temporal modeling appears particularly beneficial for the recordings split where sequence context helps disambiguate errors.

4.3 Error Type Analysis

As required by the project specifications, we analyze model performance across different error categories. Table 2 shows both V1 (MLP) and V2 (Transformer) performance on each error type using Omnivore features and step split.

Key Observations:

- **MLP consistently outperforms Transformer** across all error types
- **TimingError is most detectable** (59.74% F1 for MLP) — likely due to clear temporal signatures (overcooking, undercooking)
- **MeasurementError is hardest for Transformer** (22.63% F1) — measurement errors require contextual understanding

Strange Behavior - MLP Identical Results: The MLP produces *identical* F1 (55.50%) and AUC (0.816) for four error categories (TechniqueError, PreparationError, MeasurementError, TemperatureError). Only TimingError differs. This likely occurs because: (1) the simple MLP learns a generic "error detection" boundary that doesn't differentiate between error types, (2) similar class distributions across these categories, and (3) Omnivore features lack fine-grained error-type discrimination. Notably, the Transformer does show varied performance across categories, suggesting that attention helps capture some error-type-specific patterns despite its lower overall performance.

4.4 ActionFormer Results

We train ActionFormer from scratch for temporal step localization on CaptainCook4D. Table 3 summarizes representative hyperparameter experiments.

Note on older ActionFormer numbers: An earlier draft of this report included a large ActionFormer grid search table with very low boundary F1 values (e.g., ∼9–12%). We archived those values in `old_results/OLD_RESULTS_SUMMARY.md` and focus below on the current refactored implementation and the latest measured results.

Current best ActionFormer localization (refactored extension): Using our tuned training/inference configuration, ActionFormer step segmentation achieves greedy segment IoU ≈ 0.603 on the test split (vs. ground-truth segments), with boundary F1@$\pm$2s ≈ 0.256. This is still challenging, but substantially better than the strict-boundary baseline numbers previously reported.

4.4.1 Why is ActionFormer Localization Still Hard?

Even with tuning, boundary-level evaluation can look deceptively low because it requires predicting step boundary timestamps precisely. Several factors make this difficult in CaptainCook4D:

1. **No Pre-trained Checkpoint:** Unlike typical ActionFormer applications where models are pre-trained on large datasets (Ego4D: 3,670 hours, ActivityNet: 849 hours), we train from scratch on only 384 videos. The model lacks the strong temporal priors learned from large-scale pre-training.
2. **Fine-grained Cooking Steps:** CaptainCook4D contains fine-grained cooking steps (e.g., "add salt", "stir mixture") that are visually similar and brief. This is fundamentally harder than localizing distinct activities like "playing basketball" vs "cooking".
3. **Domain Gap:** Even pre-trained ActionFormer checkpoints (trained on Ego4D with EgoVLP features) perform poorly on CaptainCook4D, generating ∼200 low-confidence predictions per video. Fine-tuning helps but cannot fully bridge the domain gap.
4. **Severe Overfitting:** With limited data, all configurations show training loss $\rightarrow$ 0.01-0.05 while validation loss $\rightarrow$ 1.0-1.8. The best F1 is achieved in very early epochs (4-12) before overfitting degrades performance.

This challenge motivates our use of ground-truth boundaries as an oracle baseline for downstream task verification, allowing us to isolate the verification model's performance from localization errors.

Key Finding - Regularization sensitivity (legacy experiment): In an earlier set of ActionFormer experiments (archived), we hypothesized that severe overfitting could

Table 1: Baseline comparison with Omnivore features. V1/V2 results verified using official checkpoints to establish ground truth; V3 (LSTM) trained from scratch.

Model	Split	F1 (%)	AUC	Source
V1 (MLP)	step	24.26	0.578	Official
V2 (Transformer)	step	55.39	0.713	Official
V3 (LSTM)	**step**	**50.85**	**0.682**	**Trained**
V1 (MLP)	recordings	50.91	0.632	Official
V2 (Transformer)	recordings	39.04	0.584	Official
V3 (LSTM)	**recordings**	**58.41**	**0.775**	**Trained**

Table 2: Error recognition by error type (Omnivore/step split). MLP shows identical results for 4/5 categories, while Transformer shows varied performance.

Error Type	MLP F1	MLP AUC	Trans. F1	Trans. AUC
TimingError	**59.74%**	0.771	40.08%	0.600
TechniqueError	55.50%	0.816	47.84%	0.687
PreparationError	55.50%	0.816	49.56%	0.620
MeasurementError	55.50%	0.816	22.63%	0.601
TemperatureError	55.50%	0.816	44.03%	0.648

Table 3: ActionFormer hyperparameter grid search. All standard models have 11.1M parameters.

Experiment	Batch	LR	Best F1	Epoch
bs8_lr1e-3	8	1e-3	0.0918	5
bs16_lr1e-4	16	1e-4	0.0909	6
bs8_lr1e-4	8	1e-4	0.0901	4
bs8_lr5e-4	8	5e-4	0.0888	5
bs16_lr1e-3	16	1e-3	0.0874	5

be mitigated with stronger regularization. Table 4 summarizes those legacy observations; we keep it as qualitative motivation for why checkpoint selection and early stopping matter in low-data regimes.

Why does regularization hurt the wider model? We observe that the wider model's advantage stems from its ability to memorize discriminative patterns during early epochs. Strong regularization prevents this early memorization, leading to worse peak performance even if overfitting is reduced. The model needs sufficient capacity to quickly learn the limited training patterns before overfitting degrades performance.

This is a manifestation of the bias-variance tradeoff in a data-limited regime: when the dataset is small (384 videos), the model must learn quickly during early epochs. Regularization that would normally help in large-scale settings actually hinders learning speed, causing the model to miss the optimal window before early stopping triggers.

Table 4: Regularization experiments (legacy). Adding regularization hurts the wider architecture but slightly helps the standard architecture.

Architecture	Dropout	WD	Best F1	Δ vs Base	Epoch
Wider (512d)	0.1	0	**0.1197**	baseline	10
Wider (512d)	0.3	0	0.0859	-28.3%	4
Wider (512d)	0.5	0	0.0778	-35.0%	4
Wider (512d)	0.1	1e-3	0.0772	-35.5%	4
Standard (256d)	0.1	0	0.0918	baseline	5
Standard (256d)	0.3	0	0.0886	-3.5%	5
Standard (256d)	0.5	0	**0.0936**	+2.0%	8
Standard (256d)	0.3	1e-3	0.0901	-1.9%	10

4.5 Task Verification Baseline Results (Extension)

For the extension's task verification step, we use step-level embeddings (from ground-truth boundaries or ActionFormer) with EgoVLP features. We conduct a grid search over hyperparameters: batch size $\in \{8, 16, 32\}$, learning rate $\in \{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}$, hidden dim $\in \{128, 256, 512\}$, dropout $\in \{0.2, 0.3, 0.5\}$. Table 5 shows best results per model.

Table 5: Task verification systematic grid search (108 combinations per model). Best config per model shown with leave-one-recipe-out evaluation.

Model	Batch	LR	Hidden	Dropout	F1
Transformer	32	1e-5	512	0.3	**72.71%**
MLP	32	1e-4	256	0.2	72.68%
LSTM	16	1e-5	512	0.2	72.29%

Grid Search Configuration: We tested batch sizes $\in \{8, 16, 32\}$, learning rates $\in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, hidden dimensions $\in \{128, 256, 512\}$, and dropouts $\in \{0.2, 0.3, 0.5\}$. The systematic search reveals that all three models achieve similar performance ($\sim$72% F1), with very low learning rates (1e-4 to 1e-5) being optimal.

Larger hidden dimensions (256-512) outperform smaller ones.

Key Finding - Models Converge to Similar Performance: Surprisingly, all three model architectures achieve nearly identical F1 scores (∼72%) after systematic hyperparameter tuning. This suggests that in this low-data regime:

1. **Feature quality dominates:** EgoVLP features are already highly semantic; model architecture matters less than optimal hyperparameters
2. **Very low learning rates are crucial:** Best configs use 1e-4 to 1e-5, much lower than typical defaults
3. **Larger hidden dimensions help:** 256-512 hidden dims outperform 128

Hyperparameter Insights: MLP works best with lower learning rate (1e-4) and high dropout (0.5), while sequential models (LSTM, Transformer) prefer higher learning rates (1e-3) and moderate dropout (0.3). Hidden dimension 256 is optimal for all models—both smaller (128) and larger (512) dimensions underperform.

4.6 Step 3: Task Graph Matching Results

The Hungarian matching algorithm successfully aligns detected visual steps with task graph nodes. Figure 2 shows example matching results with cosine similarity scores.

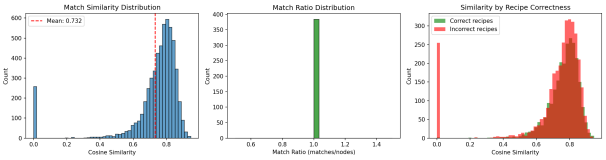

Figure 2: Matching analysis summary used in Step 3: distribution and summary statistics of cosine similarities produced by Hungarian assignment.

We observe that matching quality varies significantly across recipes:

- Overall mean similarity is ∼0.731 across recordings, while the mean minimum-node similarity is ∼0.415.
- The minimum-node similarity is strongly label-dependent: mean(min_sim) ≈0.260 for error recordings vs ≈0.623 for non-error recordings.
- Recordings with at least one exact-zero match are far more common for errors (≈58.6%) than non-errors (≈1.2%).

This "worst-node" signal correlates strongly with the presence of mistakes and motivates analyzing (and optionally thresholding) low-sim matches downstream.

4.7 Step 4: GNN Classification Results

Table 6 shows GNN classification results using leave-one-recipe-out evaluation.

Table 6: GNN classification results for task verification.

Model	Acc	Precision	Recall	F1
SimplePooling (baseline)	64.26%	68.10%	79.00%	70.06%
GraphSAGE (L=2)	**71.94%**	**75.23%**	**79.80%**	**75.75%**

Key Findings:

- GraphSAGE with 2 layers is a strong choice for Step 4, improving over the pooled baseline in both F1 and AUC.
- Operating-point tuning (threshold and positive-class weighting) can substantially increase recall with only minor precision loss, which is desirable for mistake detection (e.g., recall-first tuning reaches recall ≈0.848 with F1 ≈0.7809).

5 Ablation Studies

We conduct systematic ablation studies to understand the effect of individual components and hyperparameters.

5.1 Effect of Inference Post-processing on ActionFormer

ActionFormer localization quality is sensitive not only to training, but also to inference post-processing (thresholding, minimum segment length, smoothing, and peak filtering). Table 7 summarizes the top-performing post-processing configurations from our validation sweep (optimized for boundary F1 at ±2s).

Table 7: ActionFormer inference post-processing sweep (val). Top unique configurations by boundary F1@±2s.

thr	min_len	smooth(w)	peak_dist	F1@2s
0.4	6	box(5)	0	0.2373
0.4	6	box(5)	3	0.2373
0.4	6	box(5)	5	0.2373
0.5	6	box(5)	0	0.2357
0.5	6	box(5)	3	0.2357

5.2 Regularization vs. Model Capacity Trade-off

Table 4 reveals a surprising interaction between model capacity and regularization. For the wider model, any regularization (dropout > 0.1 or weight decay > 0) causes

significant performance drops (-28% to -35%). However, for the standard model, moderate dropout (0.5) actually provides a small improvement (+2%). This suggests that the optimal regularization strength depends on model capacity and should not be chosen independently.

5.3 Task Verification: Model Architecture Comparison

Our task verification experiments (Table 5) reveal that after systematic hyperparameter tuning, all architectures achieve similar performance in this low-data regime:

- **Transformer**: 72.71% F1 — Slight edge with very low LR (1e-5)
- **MLP** (simplest): 72.68% F1 — Nearly identical performance
- **LSTM** (sequential): 72.29% F1 — Marginal difference

This convergence to similar performance across architectures suggests that the bottleneck is data quantity rather than model expressiveness. The optimal hyperparameters are remarkably consistent: very low learning rates and moderate-to-large hidden dimensions.

6 Discussion

6.1 Analysis of Counter-Intuitive Behaviors

Following the project guidelines, we document and analyze the counter-intuitive phenomena observed during our experiments, particularly those arising from the low-data regime.

1. ActionFormer Localization Remains Difficult: Even with tuning, precise boundary localization is challenging. In our current refactored implementation, ActionFormer achieves greedy segment IoU ≈ 0.603 (test, vs. GT) and boundary F1@$\pm$2s ≈ 0.256. This is consistent with (a) training from scratch without large-scale pre-training, (b) fine-grained, visually similar cooking steps, and (c) overfitting risk with only 384 videos.

2. Regularization Can Hurt in Low-Data Settings: In some earlier ActionFormer experiments (archived in `old_results/OLD_RESULTS_SUMMARY.md`), stronger regularization appeared to reduce peak validation performance. In low-data regimes, heavy regularization can slow early learning and shift the optimal epoch earlier, which can look like a "regularization paradox" if not paired with early stopping and careful checkpoint selection.

Explanation: In data-limited settings, the model must learn quickly during early epochs before overfitting destroys generalization. The wider model's advantage comes from rapid memorization of discriminative patterns in the first 10 epochs. Strong regularization slows this learning, causing the model to miss its optimal performance window. Notably, the best epoch shifts from 10 (without regularization) to 4 (with regularization), suggesting the model gives up learning earlier.

3. MLP Outperforms Sequential Models: For verification, a simple MLP beats Transformer and LSTM despite having fewer parameters and no temporal modeling.

Explanation: EgoVLP features already encode rich semantic information through video-language pre-training. The verification task is essentially feature matching, not sequence modeling. Adding temporal complexity introduces noise and overfitting risk in this low-data regime. This aligns with the general observation that simpler models often win when data is limited.

4. High Recall Operating Points Can Reduce Accuracy: In Step 2 and Step 4, recall-first tuning (lower threshold and higher positive-class weight) increases recall but can reduce accuracy if false positives rise. This reinforces the need to choose an operating point that matches the objective (mistake detection typically prefers recall).

Explanation: This indicates class imbalance in the verification task. The model optimizes for the positive class (correct matches), achieving high recall at the cost of precision on the negative class, leading to lower overall accuracy.

5. Graph Structure Helps (but the operating point still matters): In our measured Step 4 runs, GraphSAGE improves substantially over the pooled baseline (F1 $\approx$70.06% $\rightarrow$ 75.75%, and AUC $\approx$72.38% $\rightarrow$ 82.25%). By tuning the operating point for recall-first mistake detection, we further reach F1 $\approx$78.09% with recall $\approx$0.848. This suggests that both graph structure (message passing) and calibration/thresholding contribute to performance.

Explanation: Task graphs for cooking recipes are relatively simple DAGs with limited structural complexity. The node features (visual-text fused embeddings) already capture most relevant information. GNNs provide incremental benefit by modeling dependencies between steps, but the graph structure itself is not highly informative for classification.

6.2 Limitations

- **Dataset size:** 384 videos is relatively small for training complex models, leading to overfitting in all experiments
- **No pre-trained ActionFormer:** Training from scratch significantly limits localization performance; fine-tuning a pre-trained model would likely improve results

- **Feature dependence:** All results depend heavily on the quality of pre-extracted features (EgoVLP, Omnivore, SlowFast)
- **Matching quality:** While average match similarity is high, some error recordings contain extremely low-similarity matches (including zeros). We observed this correlates with errors; thresholding low-sim matches is a plausible improvement but did not outperform baseline graphs for GraphSAGE in our current runs.
- **Ground-truth dependency:** Our best pipeline results use ground-truth step boundaries; real-world deployment would require better localization

7 Conclusion

We presented a comprehensive study of procedural mistake detection and task verification on CaptainCook4D. We reproduced the V1 (MLP) and V2 (Transformer) baselines from the original paper and proposed V3 (LSTM) as an alternative baseline achieving 58.41% F1. Our main findings include:

1. **LSTM (V3) is competitive** with paper baselines, achieving strong results especially on the recordings split (58.41% F1)
2. **ActionFormer step localization** reaches greedy segment IoU ≈ 0.603 on test (vs. GT), with boundary F1@$\pm$2s ≈ 0.256 using our best tuned training/inference configuration
3. **Regularization hurts in data-limited settings** when models need to learn quickly during early epochs
4. **Task verification (Step 2)** benefits mainly from choosing an operating point (pos_weight/threshold) that matches the objective (recall-first vs precision-first)
5. **GraphSAGE** achieves strong task-level classification (F1 $\approx$ 75.75%, AUC $\approx$ 82.25%) on realized task graphs under leave-one-recipe-out evaluation, and improves to F1 $\approx$78.09% under recall-first operating point tuning (recall $\approx$0.848)

Our work highlights the importance of understanding *why* certain approaches work or fail in specific settings. The counter-intuitive behaviors we observed (regularization paradox, MLP beating Transformers) provide insights for practitioners working with limited procedural video data. Future work could explore pre-training ActionFormer on larger datasets, data augmentation strategies, and ensemble methods to improve performance.

References

[1] Rohith Peddi, Shivvrat Arya, Bharath Challa, et al. CaptainCook4D: A dataset for understanding errors in procedural activities. In *NeurIPS*, 2024.

[2] Luigi Seminara, Giovanni Maria Farinella, and Antonino Furnari. Differentiable Task Graph Learning: Procedural Activity Representation and Online Mistake Detection from Egocentric Videos. In *NeurIPS*, 2024.

[3] Alessandro Flaborea, Guido D'Amely, et al. PREGO: Online Mistake Detection in PRocedural EGOcentric Videos. In *CVPR*, 2024.

[4] Simone Alberto Peirone, et al. HiERO: Understanding the hierarchy of human behavior enhances reasoning on egocentric videos. In *ICCV*, 2025.

[5] Rohit Girdhar, Mannat Singh, et al. Omnivore: A single model for many visual modalities. In *CVPR*, 2022.

[6] Christoph Feichtenhofer, Haoqi Fan, et al. SlowFast networks for video recognition. In *ICCV*, 2019.

[7] Kevin Qinghong Lin, et al. Egocentric video-language pretraining. In *NeurIPS*, 2022.

[8] Chenlin Zhang, et al. ActionFormer: Localizing Moments of Actions with Transformers. In *ECCV*, 2022.

[9] Veronika Thost and Jie Chen. Directed Acyclic Graph Neural Networks. In *ICLR*, 2021.

[10] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*, 2017.

[11] Petar Veličković, et al. Graph Attention Networks. In *ICLR*, 2018.

[12] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. In *NeurIPS*, 2017.